SYSTEM ARCHITECTURE

Progress Notes from Laboratory & Radiology Reports

Upload ⟶ Extraction ⟶ Structured Storage ⟶ Comparison & AI-Generated Progress Notes

1. Overview

This document describes the end-to-end architecture that turns uploaded patient reports (lab and radiology) and vitals into AI-generated progress notes. The system is organized into three phases: (1) uploading and validating source files, (2) extracting and storing structured data, and (3) retrieving that data to compare current versus historical values and generate a narrative summary. Every record created in any phase — files, structured rows, and vector embeddings — is tagged with the same two identifiers, patient_id and admission_id, which is what lets the system join data across four different stores at query time.

1.1 Four data stores, one join key

The architecture deliberately spreads data across four purpose-built stores rather than one database. Each store is used for a different kind of comparison at generation time:

- Vitals comparison — read from the PostgreSQL vitals table (time-series numeric data suits a relational store with fast range queries).
- Lab report comparison — read from the PostgreSQL patient_reports table (structured key/value lab results, also relational).
- Radiology image display — read from Blob Storage, where processed X-ray / CT / MRI images are kept as files, organized by patient_id / admission_id / report_date.
- Summary and explanation — read from the vector database, where radiology and free-text report content is embedded and semantically searched, then handed to the LLM to ground its narrative.

Because every row, file path, and embedding carries the same patient_id and admission_id, Phase 3 can pull matching records from all four stores for one patient encounter and reason over them together.

PHASE 1

Upload Reports + Vitals

Doctors / Nurses / Labs / Admin submit source files

1. Upload via Web App / API

- Upload patient reports as PDF or image files.
- Upload a vitals.csv file for the encounter.
- Every upload is tagged with patient_id and admission_id.

2. Validations

- Validate that patient_id / admission_id exist and are well-formed.
- Validate file types (PDF/Image/CSV) and reject anything else.
- Validate the internal format of vitals.csv against the expected column set.

3. Report Classification

- Detect the report type (CBC, LFT, Radiology, etc.) from the file content or metadata.

- Extract the report date and time for downstream ordering.

4. Build Blob Path

- Construct a deterministic storage path from patient_id, admission_id, report type, and timestamp so any later step can locate the file without a database lookup.

```
P001/Admission_001/
  Reports/2026-07-10/09-00AM/CBC_Report.pdf
  vitals/2026-07-10/vitals_09AM.csv
```

5. Store in Azure Blob Storage

- Persist the original report files (PDF / images) at the path built in step 4.
- Persist vitals.csv alongside the reports for the same admission.

Hand-off: *writing a new file to Blob Storage raises an event trigger that starts Phase 2 — nothing in Phase 1 calls Phase 2 directly.*

PHASE 2

Extract & Store Structured Data

Azure Function pipeline, triggered on new file upload

1. Azure Function (Trigger)

- Triggered automatically when a new file lands in Blob Storage.
- Reads the file (report or vitals.csv).
- Identifies the report type from the blob path/content.

2. Azure AI Document Intelligence

- Runs OCR on the source file.
- Detects layout and extracts tables.
- Extracts key information (patient identifiers, test names, values, dates).

3. Process Based on Report Type

- The pipeline branches here — lab reports and radiology reports are handled differently because they produce different kinds of data.

Branch A — Lab Reports (Pathology)

- Extract result tables and key/value pairs.
- Normalize and clean values (units, decimal formats, flag codes).
- Map each test to a standard unit so later comparisons are apples-to-apples.
- Return structured data for storage in SQL.

Branch B — Radiology Reports

- Extract the report text (Findings / Impression sections).
- Extract embedded images (X-ray, CT, MRI).
- Clean and chunk the text so it can be embedded for semantic search.

4. Storage — four destinations

Each branch writes to the store best suited to how that data will be read later:

4A. Store Lab Data in SQL DB

- Saved into the patient_reports table as structured lab values.
- Every row includes patient_id, admission_id, and report_type.

4B. Store Vitals Data in SQL DB

- vitals.csv is parsed and saved into the vitals table as time-series rows.
- Every row is linked to patient_id and admission_id.

4C-1. Store Radiology Text in Vector DB

- Generates embeddings for the cleaned report text chunks.
- Stores the chunks with metadata: patient_id, admission_id, report_type.

4C-2. Store Radiology Images in Blob Storage

- Saves the extracted images (X-ray, CT, MRI).
- Organized by patient_id, admission_id, and report_date.

```
P001/Admission_001/
  Radiology_Images/2026-07-10_09AM/
    Chest_Xray_1.png
    Chest_Xray_2.png
```

PHASE 3

Analysis, Comparison & Progress Notes

Reads all four stores to generate a grounded clinical summary

1. Retrieve Data

- Get lab reports and vitals from the SQL DB — both current and historical rows for the admission.
- Get radiology images from Blob Storage.
- Get radiology and report text from the Vector DB.

2. Analysis & Comparison

- Compare the current report against previous reports to compute trends.
- Analyze vitals trends over the retrieved time window.
- Run semantic search over radiology / report text in the Vector DB.
- Correlate lab, vitals, and radiology findings for the same encounter.

3. LLM (Azure OpenAI)

- Generates a clinical summary grounded in the retrieved and compared data.
- Surfaces clinical insights, risk alerts, and abnormal trends.
- Produces recommendations.

4. Generate Progress Notes

- Patient summary.
- Lab and vitals trends.
- Radiology findings, including the associated images.

- Clinical recommendations.

5. **Output / Visualization**
   - Web dashboard for doctors and nurses.
   - Trend charts and tables.
   - Radiology image viewer.
   - Export to PDF, Excel, or CSV.

2. Data Stores — Detailed Structure

The four stores below are the ones referenced throughout Phases 1–3. Folder paths and table columns are shown as they appear in the pipeline.

2.1 Azure Blob Storage — Raw Files & Images

Holds everything uploaded in Phase 1, plus the radiology images extracted in Phase 2.

```
├── P10001 |
├── Admission_001
|  |
|  ├── CBC
|  |  ├──2026-07-10
|  |  |  ├──09-00AM
|  |  |  |  P001_CBC.pdf
|  |  |  |
|  |  |  ├──01-00PM
|  |  |     P001_CBC.pdf
|  |
|  ├── BMP
|  ├── CRP

  |vitals/
      vitals_2026-07-10_09AM.csv

 |Radiology_Images/
      Chest_Xray_1.png
      Chest_Xray_2.png
   ...
```

2.2 Azure PostgreSQL Database — Structured Data

Two tables carry all structured, queryable clinical data. Both are keyed by patient_id + admission_id, which is what makes "current vs. historical" comparisons a straightforward range query.

`patient_reports` (Lab Reports)

Column	Type	Notes
id	PK	Row identifier.
patient_id	text	Join key across all four stores.
admission_id	text	Join key across all four stores.

Column	Type	Notes
report_type	text	e.g. CBC, LFT, CMP.
test_name	text	Individual lab test.
result_value	text / numeric	Normalized result.
unit	text	Standardized unit.
reference_range	text	Normal range for the test.
report_date	timestamp	When the report was collected.
collected_at	timestamp	Specimen collection time.
created_at	timestamp	Row insert time.

vitals (Time-series Vitals)

Column	Type	Notes
id	PK	Row identifier.
patient_id	text	Join key across all four stores.
admission_id	text	Join key across all four stores.
recorded_at	timestamp	When the vitals were taken.
heart_rate	integer	bpm.
systolic_bp	integer	mmHg.
diastolic_bp	integer	mmHg.
spo2	integer	%.
respiratory_rate	integer	breaths/min.
temperature	numeric	°F or °C.
created_at	timestamp	Row insert time.

2.3 Vector Database — Text Embeddings

Used for semantic search and summarization. Two collections separate radiology narrative text from other free-text report content, but share the same metadata shape so both can be queried together for a given patient_id / admission_id.

radiology_text_embeddings

Column	Type	Notes
id	PK	Chunk identifier.
patient_id	text	Join key across all four stores.
admission_id	text	Join key across all four stores.
report_type	text	e.g. Chest X-Ray, CT.
report_date	timestamp	When the report was generated.

Column	Type	Notes
text_chunk	text	Cleaned Findings / Impression text.
embedding	vector	Embedding of text_chunk.
metadata	jsonb	Additional context for retrieval.

reports_text_embeddings

Column	Type	Notes
id	PK	Chunk identifier.
patient_id	text	Join key across all four stores.
admission_id	text	Join key across all four stores.
report_type	text	e.g. CBC, LFT.
report_date	timestamp	When the report was generated.
text_chunk	text	Cleaned report text.
embedding	vector	Embedding of text_chunk.
metadata	jsonb	Additional context for retrieval.

2.4 Blob Storage — Processed Images

Radiology images extracted in Phase 2 (step 4C-2) are re-organized here by patient, admission, and report date so the dashboard can fetch "all images for this encounter" with a single prefix lookup.

```
P001/
  Admission_001/
    Radiology_Images/
      2026-07-10_09AM/
        Chest_Xray_1.png
        Chest_Xray_2.png
        CT_Scan_1.png
  ...
```

3. End-to-End Flow

Tracing one admission from upload to progress note:

1. A nurse uploads a CBC PDF and vitals.csv for P001 / Admission_001 through the web app.
2. The files are validated, classified, and written to Blob Storage under the P001/Admission_001 path.
3. The Blob write raises an event trigger; an Azure Function reads the new file and runs it through Azure AI Document Intelligence.
4. Because it’s a CBC (a lab report), the pipeline extracts the result table, normalizes units, and writes structured rows to the patient_reports table in PostgreSQL. The vitals.csv is parsed the same way into the vitals table.

5. If the upload had instead been a radiology report, the pipeline would extract Findings/Impression text — embedded and stored in the Vector DB — and the embedded images — saved to the processed-images Blob path.
6. When a doctor opens the patient dashboard, Phase 3 retrieves: current and historical rows from patient_reports and vitals (SQL), radiology images (Blob), and radiology/report text (Vector DB) — all filtered by the same patient_id and admission_id.
7. The vitals and lab comparisons are computed directly from the SQL rows (current value vs. prior value, in-range vs. out-of-range). Radiology images are rendered directly from Blob Storage.
8. For narrative summary and explanation, the Vector DB is searched semantically for the text most relevant to the current findings, and that retrieved text — together with the SQL comparisons — is passed to Azure OpenAI to generate the progress note.
9. The dashboard renders the resulting summary, trend charts, tables, and radiology images, with export to PDF / Excel / CSV.

3.1 Store-to-purpose summary

Data need	Source store	Why
Vitals comparison	PostgreSQL — vitals table	Time-series numeric data; fast range queries by patient_id/admission_id.
Lab report comparison	PostgreSQL — patient_reports table	Structured key/value results; direct current-vs-previous row comparison.
Radiology image display	Blob Storage — processed images	Large binary files are cheapest and fastest to serve directly from blob storage.
Summary & explanation	Vector DB — text embeddings	Semantic search surfaces the most relevant narrative text to ground the LLM's explanation.

4. User Interfaces

- Web Dashboard — primary interface for doctors and nurses; trend charts, tables, radiology viewer, exports.